

Interaktive Webkarten mit Vectortiles und MapLibre

Jakob Listabarth
27.05.2026
74. DKK, Dresden

Guten Morgen!



Jakob Listabarth

- PhD-Student an der TU Dresden
 - Fokus: Schematische Karten
- Grafischer Hintergrund und Zugang zur Kartografie

Inhalt

Hintergrund

Setup

Style Specifications

Offene Daten integrieren

Marker und Interaktionen

Mini-Projekt

Appendix

Hintergrund

Was ist MapLibre?



- open source fork von mapbox-gl-js (December 2020)
- verwendet *WebGL*-Technologie (GPU) → performant
- entwickelt speziell für vectortiles

Was sind Vectortiles?

- Speicherformat für mathematische Beschreibungen (Vektoren) geometrischer Formen (Punkten, Linien, Polygonen) und deren Attributen (Daten) in quadratischen Kacheln
- Beinhalten nur die geometrischen Informationen (und Attribute) und **nicht das Aussehen**
- Erst der Browser rendert die Daten zu einer Karte basierend auf Stilen

Links

- Kacheln (Quadtree): [Vectortiles](#)
- Raster- vs. Vectortiles: [Basemap.de Viewer](#)

Vectortiles: Vor- und Nachteile

Vorteile

- dynamisch und anpassbar
 - Accessibility: Schriftgröße, Kontrast, Farben
 - Sprache (Internationalisierung)
 - Inhalte (Filtern in Stil-Definition)
- kontinuierlicher Zoom
- geringere Speichergröße und weniger komplex → mehr Angebote als Endnutzer*in

Nachteile

- benötigt relative leistungsstarke Geräte (WebGL)
- Qualitätsverlust: weniger Details, Auflösung, Anti-Aliasing
- Begrenzt in Bezug auf visuelle Details oder Stile (z. B. keine Schatten, Verläufe etc.)
 - lösbar durch custom shaders (komplex!)

Web Kontext 🧑🏻💃🏻🌐

Wäre das *World wide web* eine Tanzfläche, dann wäre ...



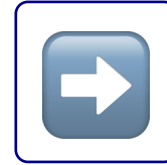
HTML
Hyper**T**ext **M**arkup
Language

... das Skelett



CSS
Cascading **S**tyle **S**heets

... das Outift



JavaScript

... die Dance-Moves

Visual Studio Code

- IDE (Integrated Development Environment), ein Texteditor
- bietet viele Funktionen, die die Entwicklung von Software erleichtern (z. B. Syntaxhervorhebung, Autovervollständigung, Debugging-Tools, etc.)
- unterstützt viele Programmiersprachen und Frameworks durch Erweiterungen (Extensions)
- kostenlos und open source

Setup

Code-Along: **HTML** Dokument aufsetzen

Das `index.html` file enthält den Code der gesamten Interaktiven Web-Karte: hier läuft alles zusammen – die Struktur (HTML), das Aussehen (CSS) und die Funktionalität (JavaScript).

Code-Along: Bibliotheken laden und Karte initialisieren

- Kartenelement vorbereiten: `<div id="map"></div>`
- Bibliothek und Styles im `<head>` laden
- Karte initialisieren

Aufgabe Optionen der Karte verändern

- Setzen Sie die Anfangsposition der Karte auf die Koordinaten von dem Ort aus dem Sie angereist sind. (Wenn Sie aus Dresden sind, dann wählen Sie den Ort der letzten DKK, Frankfurt).

Hinweis Reihenfolge der Koordinaten: [Längengrad, Breitengrad]

- Passen Sie ggfs. das Zoom-Level an.

Aufgabe Optionen der Karte verändern

- Deaktivieren Sie das Zoomen durch einen Doppelklick, finden Sie dazu die entsprechende Option in der [Dokumentation](#)

Style Specifications

Click-along: Styles anpassen

- Styles.json: Struktur und Aufbau verstehen
- Grünflächen-Layer hinzufügen

Aufgabe Maputnik 1

- Passen Sie die Farbe der Elbe an: diese soll für unsere spezifische Karte mehr heraustechen.
 - Den entsprechenden Layer (bzw. deren ID) finden Sie am einfachsten wenn sie im „Inspect“-Modus auf die Elbe klicken
 - Das Anpassen der Farbe müssen Sie das **paint** Attribute des entsprechenden Layers anpassen

Click-Along: Eigene Icons einbauen

- `sprites` hinzufügen
- `icon` im entsprechenden Layer verwenden

Aufgabe Maputnik 2

- Fügen sie Icons für Geschäfte hinzu, die in den Vectortiles als Punkte enthalten sind.
 - der `source_layer` heißt `poi` (*points of interest*)
 - Die *Phosphor* Icons finden Sie auf <https://phosphoricons.com>
 - Filter Bedingung: `class` entspricht dem Wert "`shop`"

Code Along: Style.json Datei einbinden

- in Maputnik auf *Save* klicken
- Dateinamen ggfs. anpassen und in den Ordner des `index.html` files speichern.
- Dateinamen als Wert für die `style` Option der Karte angeben

Hinweis Damit das Einbinden des lokalen Styles funktioniert, muss ein (lokaler) Webserver laufen.

*Offene Daten
integrieren*

Code-Along: GeoJson Daten einbinden

- Das GeoJson File kann entweder lokal oder von einem Server geladen werden
 - Source zu Karte hinzufügen (`type` und `url`)
 - Apotheken Dresdens: <https://kommisdd.dresden.de/net4/public/ogcapi/collections/L458/items>
 - Layer zu Karte hinzufügen
 - `paint` und `layer`-attribute anpassen
 - icons von sprites verwenden (ggfs. sprite hinzufügen)

Aufgabe GeoJson File einbinden

- Ein GeoJson File mit den Standorten aller Dresdner Freibäder ist im GitHub Repository verfügbar
 - Download unter <https://github.com/jakoblistabarth/dkk-maplibre-workshop> und lokal speichern
 - Verwenden Sie für den passenden Layer type für die Punktdaten (z. B. [circle](#), oder [symbol](#))
 - sowie entsprechende [layout](#) und [paint](#) Attribute

Code-Along: Elevation

- Vorbereitung: Karte über Sächsische Schweiz zentrieren
 - Rathen: `[14.079849, 50.957274]`
 - opt. Pitch und Zoom-Level anpassen
- `terrain`-Quelle hinzufügen (terrarium format von aws)
- Terrain-Controller oder `setTerrain` verwenden, um das Terrain zu aktivieren

Code-Along: Hillshade Layer

- mit der Terrain-Datenquelle können wir auch einen Hillshade Layer hinzufügen, um die Höhenunterschiede besser sichtbar zu machen
- Hillshade Layer hinzufügen
 - `type = hillshade`
 - `source`: Terrain-Quelle
- Layer unter labels layer platzieren

Marker und Interaktionen

Code-Along: Marker manuell hinzufügen

- einzelnen Marker zu Karte hinzufügen
 - `lat` Koordinaten verwenden
 - optional Farbe definieren
- Popup erstellen und direkt zu Marker hinzufügen

Code-Along: Popup für POIs

- Ziel: Wenn auf einen POI Layer geklickt wird, soll ein Popup mit Informationen zum POI angezeigt werden
- Funktion definieren die ausgeführt wird, wenn auf einen POI Layer geklickt wird
 - `click` Event auf Layer
- Popup erstellen und auf Koordinaten des geklickten Features platzieren
- Cursor ändern, wenn über ein Feature des POI Layers gehovert wird

Mini-Projekt

Aufgabe Mini-Projekt – Ideen

- Erstellen Sie in einer neuen HTML Datei ([project/index.html](#)), eine Karte von Dresden:
 - Stellen Sie die Standorte des DKK in einer Karte dar:
 - Ein GeoJson File mit den Standorten der Konferenz ist im GitHub Repository verfügbar
 - Download unter <https://github.com/jakoblistabarth/dkk-maplibre-workshop> und lokal speichern
 - Wählen sie einen passenden Stil von *OpenFreeMap* oder passen sie den Stil an

- die Standorte der Konferenz sollen mit einem Popup versehen werden, der den Namen des Standorts anzeigt
- binden Sie weitere Daten z. B. vom Open Data Portal Dresden ein

Appendix

Vielen Dank für Ihre Aufmerksamkeit!

Fragen? Anregungen?

jakob.listabarth@tu-dresden.de

Ressources / Github Repository



Den Code sowie das Handout finden Sie auf GitHub:
<https://github.com/jakoblistabarth/dkk-maplibre-workshop/>

HTML5 Boilerplate

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
<!-- Hier wohnt der sichtbare Inhalt der Website -->
</body>
</html>
```